

Pipelining and Hazards

COMS10015

Dr Tom Deakin

University of Bristol

Week 16

Pipelining

Motivation

- ▶ So far, processors process instructions (Fetch/Decode/Execute) one at a time.
- ▶ If one instruction takes many cycles, it introduces long delays into the processor.
- ▶ Lots of independent processor parts are idle (e.g., fetch and execute).
- ▶ Need to optimise micro-architecture to keep busy.

Note: Not all functions use same number of cycles.

Some units, such as DIV, require many cycles to complete.

Example: Pipelining Laundry (1)

Consider doing three loads of laundry.

Each load has three steps:

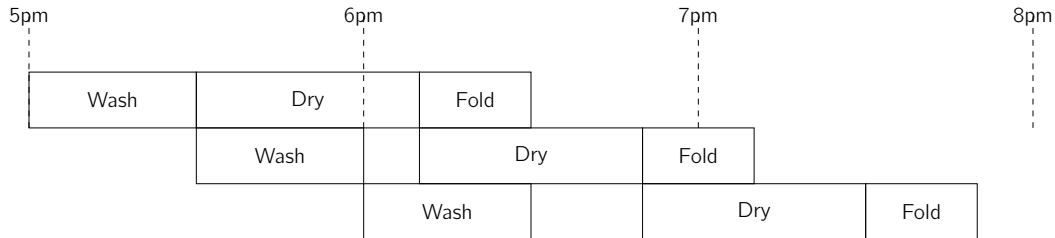
1. Put through washing machine (30 mins).
2. Put through tumble dryer (40 mins).
3. Fold clothes (20 mins).



Doing them one at a time would take 4.5 hours!

Example: Pipelining Laundry (2)

This time, put the second load in the wash when the first load is in the dryer.



Now only takes 2 hours 50 minutes!

Improved **throughput** by using all the machines simultaneously.

RISC Instruction execution cycle

A typical RISC architecture operates in these five stages:

Fetch Get an instruction from memory.

Decode Decode instruction and read operands from registers.

Execute Execute instruction or calculate memory addresses.

Memory Access memory (loads/stores).

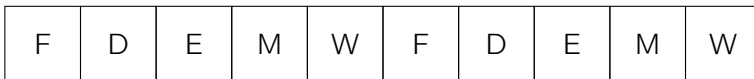
Writeback Save results back to the registers.

Each stage has dedicated hardware/logic — seen this in Hex 8 micro-architecture.

Registers used between stages to hold data associated with instruction — e.g. *rreg* in Hex 8 implementation.

Instruction execution cycle idling

- ▶ Processor performs these steps in a loop.
- ▶ Parts of the processor only active in their corresponding stage.
- ▶ Fetch logic idle during Decode/Execute/Memory/Writeback.
- ▶ Most of the execution units in the data path idle, as only those relevant to one instruction active.

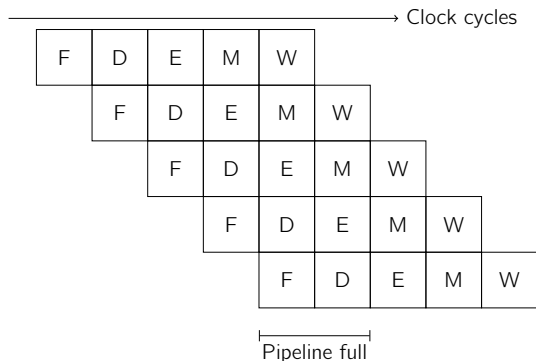


Instruction Pipelining

Instruction Pipelining

Work on different parts of the execution cycle concurrently.

- ▶ Every cycle, a new instruction enters the pipeline and a completed one leaves.
- ▶ Can use all the parts of the processor at the same time.
- ▶ Increases **throughput** of the processor.
- ▶ Ideally pipeline becomes full, performing all five steps for multiple cycles.



Long latency instructions

Some instructions inherently slow (require many cycles) such as divide and square roots.

- ▶ Slowest step determines throughput.
- ▶ Delays in the pipeline are called *bubbles* or *stalls*.
- ▶ Smart compilers help eliminate or transform divides.
- ▶ Sometimes can replace long instructions with fast ones.

Sometimes pipeline stalls are inevitable based on the specific instruction ordering.

Example: fast reciprocal square root

Case study: fast reciprocal square root in Quake III Arena

In graphics, often need to calculate square roots. This was relatively *slow* to do in hardware (1999).

Game used a fast approximate version, using multiplies, additions, subtractions and shifts.

Today, often find support for fast approximate square roots in *hardware*.

More info: https://en.wikipedia.org/wiki/Fast_inverse_square_root

Example: Calculating average time for Laundry

Back to the pipelined laundry example.

We have three loads, and each load of laundry takes 90 minutes ($30 + 40 + 20$).

- ▶ Without pipelining, it took 4 hours 30 minutes (270 minutes). That's $270/3 = 90$ minutes per load of laundry.
- ▶ With pipelining it took 2 hours 50 minutes (170 minutes). That's $170/3 = 56.7$ minutes per load of laundry.

Can use this intuition to define a metric.

Cycles per Instruction

Definition: Cycles per Instruction

Cycles per Instruction is the average number of cycles taken by each instruction in a program (or part of a program).

$$CPI = \frac{\sum_i \text{Instruction}_i \times \text{Cycles to execute Instruction}_i}{\text{Number of instructions}}$$

Useful metric for measuring performance.

Note:

A pipelined processor should get $CPI = 1$, without bubbles/stalls.

- ▶ I.e. One instruction executes every cycle.

Example: Calculating CPI

The following instructions in a pipelined processor each take a certain number of cycles:

Instruction	Cycles	Frequency
Load	6 cycles	10
Store	5 cycles	10
Add	1 cycle	20
Branch	3 cycles	8

Example: Calculating CPI

The following instructions in a pipelined processor each take a certain number of cycles:

Instruction	Cycles	Frequency
Load	6 cycles	10
Store	5 cycles	10
Add	1 cycle	20
Branch	3 cycles	8

$$CPI = \frac{6 \times 10 + 5 \times 10 + 1 \times 20 + 3 \times 8}{48} = 3.2$$

Example: Calculating CPI

The following instructions in a pipelined processor each take a certain number of cycles:

Instruction	Cycles	Frequency
Load	6 cycles	10
Store	5 cycles	10
Add	1 cycle	20
Branch	3 cycles	8

$$CPI = \frac{6 \times 10 + 5 \times 10 + 1 \times 20 + 3 \times 8}{48} = 3.2$$

This instruction mix on that processor is not able to execute one instruction per cycle.

- Lots of long latency instructions creating bubbles.

Iron Law of Processor Performance

$$\text{Execution Time} = \frac{\text{Seconds}}{\text{Program}} = \frac{\text{Instructions}}{\text{Program}} \times \frac{\text{Clock cycles}}{\text{Instruction}} \times \frac{\text{Seconds}}{\text{Clock cycle}}$$

- ▶ Instructions per Program is factor of ISA design (and compilers).
- ▶ Cycles per Instruction is factor of micro-architecture design.
- ▶ Seconds per Clock cycle is factor of chip manufacturing.

Optimising processors

CISC ISAs reduce Instructions per Program, but can increase Cycles per Instruction.
Pipelining and other micro-architectural techniques reduce Cycles per Instruction.
We have optimised the final term up to a point, but ...

The Power Wall

Final term is inverse clock frequency (Hz) which is limited by the *power wall*.

Power per transistor is proportional to the product of frequency, squared voltage and capacitive loading.

By increasing the clock speed and reducing the voltage (Dennard scaling), we can get more performance than more power used . . .

. . . unless reducing the voltage causes leakage making transistors ineffective.

Iron Law to calculate program runtime

For example, with a program with 1000 instructions, each taking 2 cycles, with a 1 GHz processor:

$$\begin{aligned}\text{Execution Time} &= \frac{\text{Instructions}}{\text{Program}} \times \frac{\text{Clock cycles}}{\text{Instruction}} \times \frac{\text{Seconds}}{\text{Clock cycle}} \\ &= 1000 \times 2 \times \frac{1}{1 \times 10^9} \\ &= 2 \times 10^{-6} \\ &= 0.000002 \text{ seconds}\end{aligned}$$

Iron Law to calculate program runtime

Or previously we calculated CPI for a program: $CPI = 3.2$. Let's say this is also a 1 GHz processor.

$$\begin{aligned}\text{Execution Time} &= \frac{\text{Instructions}}{\text{Program}} \times \frac{\text{Clock cycles}}{\text{Instruction}} \times \frac{\text{Seconds}}{\text{Clock cycle}} \\ &= 48 \times \frac{6 \times 10 + 5 \times 10 + 1 \times 20 + 3 \times 8}{48} \times \frac{1}{1 \times 10^9} \\ &= 48 \times \frac{154}{48} \times \frac{1}{1 \times 10^9} \\ &= 1.54 \times 10^{-7} \\ &= 0.000000154 \text{ seconds}\end{aligned}$$

Hazards

Hazards

Definition: Hazard

A hazard causes the instruction pipeline to stall.

That is, the hazard means the next instruction cannot execute in the next cycle.

Hazards

Three main types of hazards:

Data hazards two instructions in different pipeline stages modify the same data.

Structural hazards two instructions need the same resource.

Control hazards pipeline stalls due to branches.

Data dependencies

One instruction might be dependent on the result of another.

Example: Arm ADD Rd, Rn, Rm instruction, which computes $Rd \leftarrow Rn + Rm$.

```
ADD r3, r2, r1  (r3 = r2 + r1)
```

```
ADD r4, r3, r4  (r4 = r3 + r4)
```

There is a data dependency on r3 register.

Data dependencies

- ▶ Recall we have 5 stages: Fetch/Decode/Execute/Memory/Writeback.
- ▶ Result of first instruction `r3` not available at end of Execute cycle.
- ▶ Will be available after the Writeback stage.
- ▶ Second instruction *cannot* begin executing on the next cycle — needs to wait for Writeback (or micro-architecture optimisation we skip over).

Types of data dependency (1)

We have two instructions, I1 and I2. The instructions are fetched in that order.

Read after Write (RAW)

I2 reads the data updated by I1 before it is ready.

Known as a *true dependency* or *flow dependency*.

- ▶ The second instruction must see the updated value.
- ▶ Previous slide showed an example.

Types of data dependency (2)

We have two instructions, I1 and I2. The instructions are fetched in that order.

Write after Read (WAR)

I2 updates the data before it is read by I1.

Known as an *anti-dependency*.

```
ADD r1, r2, r3    (r1 = r2 + r3)
```

```
ADD r3, r2, r4    (r3 = r2 + r4)
```

Occurs with concurrent execution or by lack of available registers.

Types of data dependency (3)

We have two instructions, I1 and I2. The instructions are fetched in that order.

Write after Write (WAW)

Both I1 and I2 updates the data, so the order of instructions determines the final result.

Known as an *output-dependency*.

```
ADD r1, r2, r3  (r1 = r2 + r3)
```

```
ADD r1, r4, r5  (r1 = r4 + r5)
```

Occurs with concurrent execution or by lack of available registers.

Dealing with data dependencies

The processor must resolve the data dependencies.

Processor could take one of these measure:

- ▶ Delay the second instruction causing a bubble in the pipeline.
- ▶ Allow another instruction to execute first (called out-of-order, discussed in Advanced Computer Architecture course).
- ▶ Change the instruction to use a different register (register renaming).
- ▶ Provide a *forwarding* path of the result to allow execute stage output to be used as input in next execute stage immediately.

Branches

What about conditional branch instructions?

- ▶ Conditional branches change the program counter based on the data.
- ▶ E.g. BRZ and BRN Hex 8 instructions.
- ▶ Not known which is the next instruction to fetch.
- ▶ Wrong instructions fetched beyond the branch are discarded, causing a pipeline bubble.
- ▶ Branches are very common in code, so hardware added to help.
- ▶ *Branch predictors* try to predict the outcome of the branch to help keep the pipeline full.

Spectre/Meltdown, again

This was the problem with Spectre/Meltdown.

- ▶ Instructions are pipelined, so some start executing before result of branch/jump know.
- ▶ Branch predictor used to predict which instructions to fetch.
- ▶ When the branch *was not* taken, those instructions needed to be 'undone'.
- ▶ The affected processors didn't do that perfectly, which leaked information that allowed an exploit to occur.



Taking stock

- ▶ Pipelining allows parts of the processor to be used every cycle.
- ▶ Instructions are fetched every cycle, with Decode/Execute/Writeback happening concurrently.
- ▶ In ideal case, one instruction executed every cycle.
- ▶ Slowest operation determines throughput.
 - ▶ Sophisticated micro-architecture techniques can help here (see Advanced Computer Architecture course).
- ▶ The pipeline can sometimes stall, reducing throughput.

Further Reading

- ▶ Instruction pipelining: https://en.wikipedia.org/wiki/Instruction_pipelining
- ▶ Pipelining: <https://cs.stanford.edu/people/eroberts/courses/soco/projects/risc/pipelining/index.html>
- ▶ Pipelining: <http://euler.ecs.umass.edu/ece232/pdf/11-PipeliningI-11.pdf>
- ▶ L14. Pipelining: <https://safari.ethz.ch/digitaltechnik/spring2018/doku.php?id=schedule>
- ▶ Branch predictor: https://en.wikipedia.org/wiki/Branch_predictor
- ▶ Reciprocal square root: <https://medium.com/hard-mode/the-legendary-fast-inverse-square-root-e51fee3b49d9>

Further Reading

- ▶ Instruction pipelining:
https://en.wikipedia.org/wiki/Instruction_pipelining
- ▶ CPI: https://en.wikipedia.org/wiki/Cycles_per_instruction
- ▶ http://home.ku.edu.tr/comp303/public_html/Lecture7.pdf
- ▶ https://en.wikipedia.org/wiki/Iron_law_of_processor_performance
- ▶ https://en.wikipedia.org/wiki/Dennard_scaling
- ▶ https://en.wikipedia.org/wiki/Iron_law_of_processor_performance
- ▶ <https://ece757.ece.wisc.edu/lect03-752-review.pdf>